Adaptive Forest-based State Aggregation for Reinforcement Learning

1. Research Motivation

In reinforcement learning (RL), accurately approximating the value function over a continuous state space is crucial. Classical tabular approaches struggle with scalability, while neural function approximators often suffer from instability, particularly in environments with sparse rewards.

The original work by Hwang et al. introduced **Adaptive State Aggregation** using **decision trees** to discretize the state space based on learning dynamics, notably the **temporal difference (TD) error**. While this method adaptively refines the representation and offers interpretability, decision trees can overfit or suffer from limited generalization.

To overcome this, we propose an extension using **Adaptive Forest Aggregation**—a random forest-style ensemble of decision-tree-based learners—aiming to improve robustness, exploration, and generalization in RL tasks.

---

2. Literature Review

Traditional RL methods for function approximation fall into two categories:

- **Neural approximators** (e.g., DQN, PPO) offer high expressiveness but often require tuning and are data-hungry.

- **Discretization-based methods** (tile coding, uniform grids, decision trees) offer interpretability and stability, but suffer from rigidity or insufficient expressiveness.

Hwang et al. (2012) proposed a decision tree that grows and splits nodes based on error statistics (mean and variance of TD errors), using **Monte Carlo Tree Search (MCTS)** for exploration. This allows dynamic refinement of the state space, effectively addressing the curse of dimensionality without uniform grids.

However, single decision trees are prone to local overfitting and variance in estimation. **Random forests**, proven in supervised learning for their robustness and ensemble generalization, offer a compelling direction for improvement in RL.

---

3. Methodology and Approach

3.1 Adaptive State Aggregator (Original)

- Uses a decision tree to partition the state space. Each leaf node represents a subspace of the sensory input vector. Only one action can be taken in a leaf node.
- The problem is modeled as a **Semi-Markov Decision Process (SMDP)**. The key difference from a Markov Decision Process (MDP) is that a transition from one state to another can occur based on **a variable amount of time that depends on the chosen action and current state.**
- Splitting is triggered by low TD error mean and high variance in the Q values..
- Splits are made based on **t-statistics** comparing state features from positive vs. negative TD error groups.
- MCTS-based action selection with UCB encourages exploration.
- Pruning is applied to reduce overfitting based on **reliability** (confidence from usage) and **activity** (recent usage).

3.2 Adaptive Forest Agent (Proposed)

- Composed of an ensemble of independent Adaptive State Aggregators.
- Each tree is trained on a **random subset of features** (akin to random forests).
- Trees are trained independently and operate on projected versions of the environment
- Final action is selected by **aggregating Q-values** from all trees and finding the argmax.
- The Q value update for a leaf node in the tree remains the same as given below:

$$\overline{Q}_{t+\tau}(s_t,a) = (1-\alpha)\overline{Q}_t(s_t,a) + \alpha[Q(s_t,a) + Q(s_{t+1},a) + \ldots + Q(s_{t+\tau-1},a)]/\tau$$

- However since we have moved from decision trees to Random Forests the Q-values are aggregated over all trees.

$$Q_{forest}(s,a) = \frac{1}{N}\sum_{i=1}^{N} Q_i(s_i,a)$$

---

4. Experiments

Environments

- **CartPole-v1**: Balanced pole on cart, dense reward, relatively easy.
- **Acrobot-v1**: Swing a double pendulum to goal, sparse rewards, harder dynamics.

Evaluation Metric

- **Mean episodic reward** over training episodes.
- Additional metric: **Inference reward** every 10 episodes to assess policy quality in exploitation.

Hyperparameters

- Shared across agents unless specified:

  CartPole

  - Learning rate: $\alpha = 0.7$
  - Discount factor: $\gamma = 0.99$
  - Exploration rate: UCB, c=2.0
  - α decay: 0.0015
  - Split thresholds: `l_th = 100`, `th_mu = 0.5`, `th_sigma = 0.03`

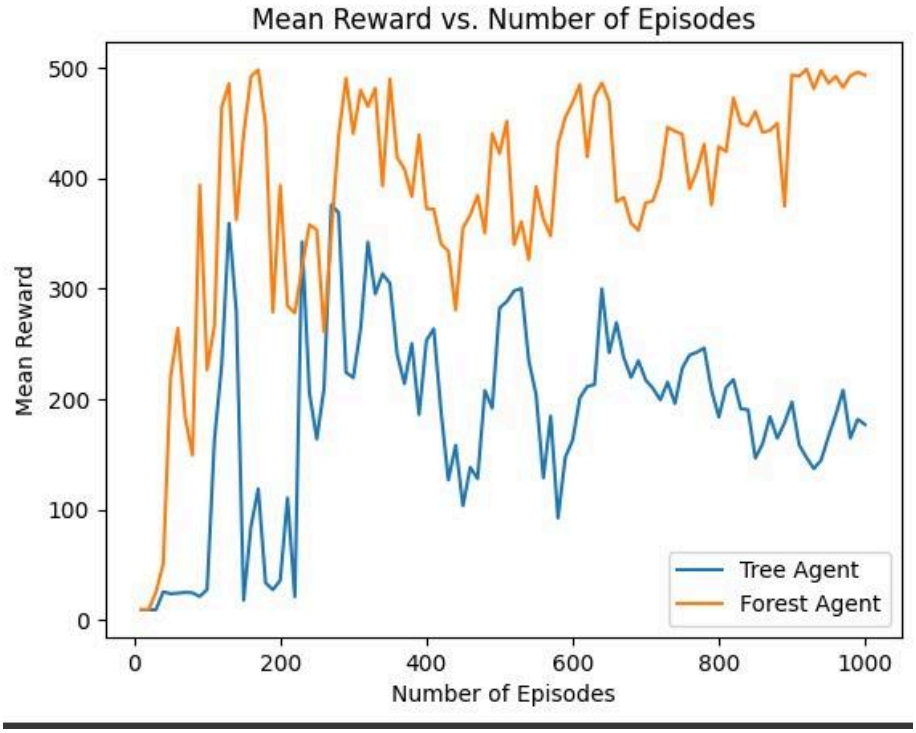
  Acrobot

  - Learning rate: $\alpha = 0.7$
  - Discount factor: $\gamma = 0.99$
  - Exploration rate: UCB, c=2.0
  - Alpha decay: 0.001
  - Split thresholds: `l_th = 100`, `th_mu = 5`, `th_sigma = 1`

---

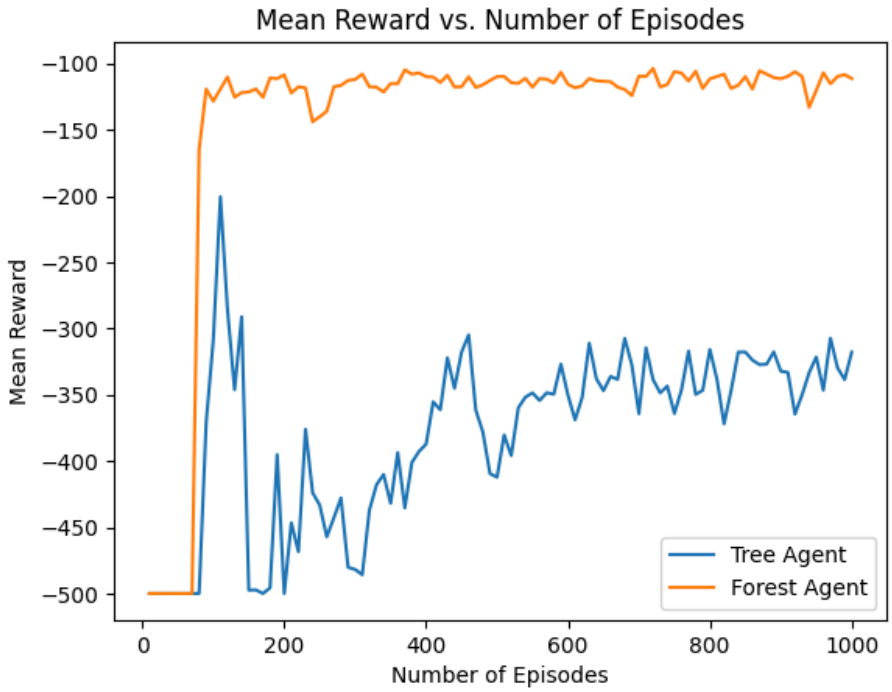
5. Results

CartPole (Dense Reward)



- **Adaptive Tree** and **Forest** agents both learn rapidly.
- Forest achieves marginally smoother and faster convergence.
- Final reward obtained in forest agent approaches the environment cap (~500), indicating optimal policy. While the tree agent fails to improve after obtaining a reward around 200. In fact, the tree agent observes a decrease in performance on further training. This maybe due to unnecessary splitting of nodes.
- **Conclusion**: For well-behaved environments, both agents perform decently, but the forest agent performs significantly better than the tree agent.

Acrobot (Sparse Reward)



- Tree agent shows limited improvement; reward remains near -300.
- Forest agent improves significantly more than the tree agent:
    - Higher mean reward by the end (~-100 to -150).
    - Better generalization likely due to ensemble learning and feature diversity.
- **Conclusion**: Forests clearly outperform single trees in sparse-reward environments, benefiting from better state representation.

  The table below shows the relation between number of trees and the reward obtained. Increasing the number of trees in general improves the performance of the model but stagnates upon reaching 8 trees. Another point to note is that the jump of the mean reward from 3 trees to 4 trees is significant.

Trial results :

Number of Trees	Mean Reward
1	-393.08 +/- 100.96
2	-360.36 +/- 105.29
3	-365.62 +/- 99.85
4	-130.62 +/- 61.28
5	-135.9 +/- 61.07
6	-123.66 +/- 31.28
7	-124.72 +/- 28.16
8	-110.32 +/- 28.87
9	-108.5 +/- 38.97
10	-112.02 +/- 39.82

6. Analysis

Strengths of Adaptive Forest:

- Better generalization across varied states.
- More robust to overfitting due to randomization.
- Handles partial observability (due to feature subsampling) well.

Weaknesses:

- Slightly more computational cost.
- Potential for instability if trees diverge too much in policy.
- Hyperparameter tuning: Finding the right parameters to tune is slightly difficult.

Key Observations:

- Ensemble method improves over original tree-based approach especially in **complex or sparse** environments.
- In simple, dense environments, both perform comparably.
- The split criterion remains central to success—further improving feature selection or split logic could boost performance.

---

7. Conclusion

This work demonstrates that **extending adaptive state aggregation from decision trees to forests** meaningfully enhances RL performance, especially in challenging domains like **Acrobot**. By leveraging random feature subsets and ensemble aggregation, the **Adaptive Forest Agent** achieves improved robustness, exploration, and reward convergence.

The method maintains interpretability and adaptability, making it a practical alternative to neural approaches in low- to mid-dimensional control problems.

---

8. Citations

1. K. -S. Hwang, Y. -J. Chen and W. -C. Jiang, "Adaptive state aggregation for reinforcement learning," 2012 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Seoul, Korea (South), 2012, pp. 2452-2456, doi: 10.1109/ICSMC.2012.6378111. keywords: {Learning;Decision trees;Estimation error;Vectors;Reliability;Monte Carlo methods;Markov processes;reinforcement learning;decision tree;MCTS},
2. Cutler, Adele & Cutler, David & Stevens, John. (2011). Random Forests. 10.1007/978-1-4419-9326-7_5.